

2D ZOMBIE SURVIVAL GAME

WEEK 10 — FINAL DOCUMENTATION & PRESENTATION

Final Testing • Debugging • Screenshots • Project Summary

Project	2D Zombie Survival Game
Technology	Java Swing
Window Size	800 × 600
Final Phase	Week 10

1. Week 10 Overview

Week 10 is the final phase of the 10-week development plan. The main purpose of this phase was to complete final testing and debugging, verify the implemented gameplay systems, capture screenshots, and prepare the project for final documentation and presentation. The final source code was reviewed to confirm that the planned gameplay features were implemented in the Java Swing project.

2. Final Testing and Debugging

The final project was reviewed against the features developed throughout the previous weeks.

Feature	Final Verification	Status
Game startup	The Main class creates the JFrame, adds GamePanel, sets the window size through the panel's preferred size, and displays the game.	Verified
Player movement	W, A, S and D movement is handled through KeyListener input, with boundary checks keeping the player inside the game area.	Verified
Zombie spawning	Zombies are continuously created using a Swing Timer. Their positions are randomized within the game area.	Verified
Zombie movement	Each zombie moves toward the current player position.	Verified
Shooting	SPACE creates a Bullet at the player's right side and bullets move horizontally across the screen.	Verified
Collision detection	Bullet-zombie and zombie-player collisions are checked using Rectangle intersection.	Verified
Score	A successful bullet-zombie collision removes both objects and increases the score by 1.	Verified
Health	Zombie-player collision decreases the player's health. Health is displayed through the HUD.	Verified
Dynamic difficulty	Every 10 points, the zombie spawn delay is reduced by 300 ms, with a minimum delay of 500 ms.	Verified
Game Over	When health reaches 0, the Game Over state is displayed and zombie spawning stops.	Verified
Restart	Pressing R after Game Over creates a new player, clears zombies and bullets, resets the score and restores the spawn delay.	Verified

3. Final Gameplay Features

- 800 × 600 Java Swing game window with a dedicated GamePanel.
- Player character with health, speed, size, drawing, movement and collision boundary.
- Four-direction keyboard movement with screen boundary protection.
- Continuously spawning zombies with randomized spawn positions.
- Zombie AI that moves toward the player.
- Bullet creation and horizontal movement using the SPACE key.
- Rectangle-based collision detection between bullets and zombies.
- Score increases when a zombie is eliminated.
- Health bar and score display in the game HUD.
- Gradually increasing zombie spawn speed as the score reaches multiples of 10.
- Game Over overlay with final score.
- Restart system using the R key.
- Anti-aliased rendering and a custom game background/grid for improved presentation.

4. Final Controls

Key	Function
W	Move Up
S	Move Down
A	Move Left
D	Move Right
SPACE	Shoot Bullet
R	Restart after Game Over

5. Dynamic Difficulty Verification

The final GamePanel implementation starts zombie spawning with a 2.0-second delay. Whenever the score becomes a multiple of 10, the delay is reduced by 300 milliseconds. The delay is limited to a minimum of 0.5 seconds, preventing the spawning system from becoming excessively fast.

Score Range	Spawn Interval
0–9	2.0 seconds
10–19	1.7 seconds
20–29	1.4 seconds
30–39	1.1 seconds
40–49	0.8 seconds
50+	0.5 seconds minimum

6. Final Gameplay Screenshot

The following screenshot represents the completed gameplay interface, including the player, multiple zombies, HUD, score display, health bar, game area and visual background.

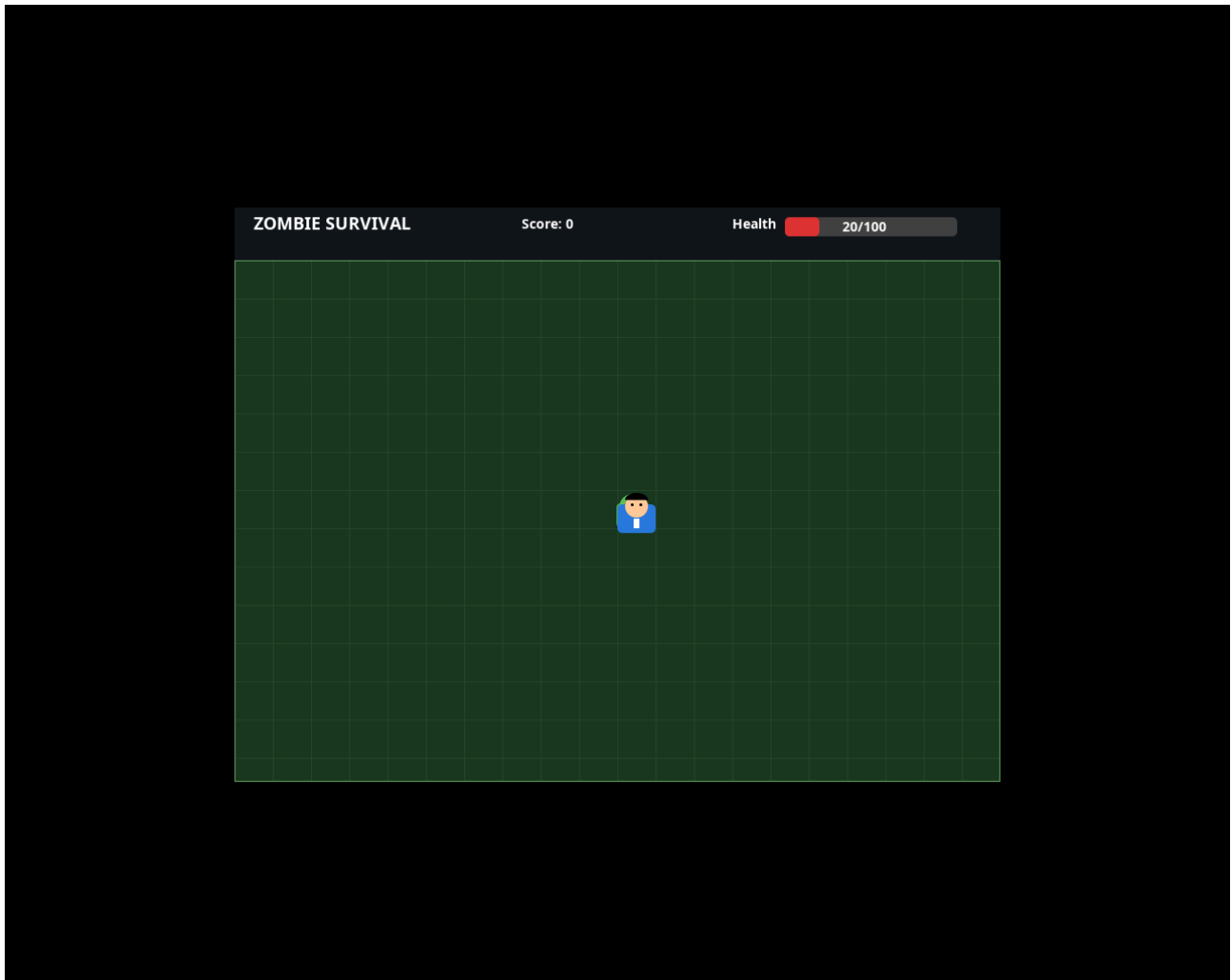


Figure 1: Final gameplay screen with active zombies and HUD.

7. Game Over Screenshot

The final project includes a dedicated Game Over state. When the player's health reaches zero, the game stops normal gameplay and displays the final score together with the restart instruction.

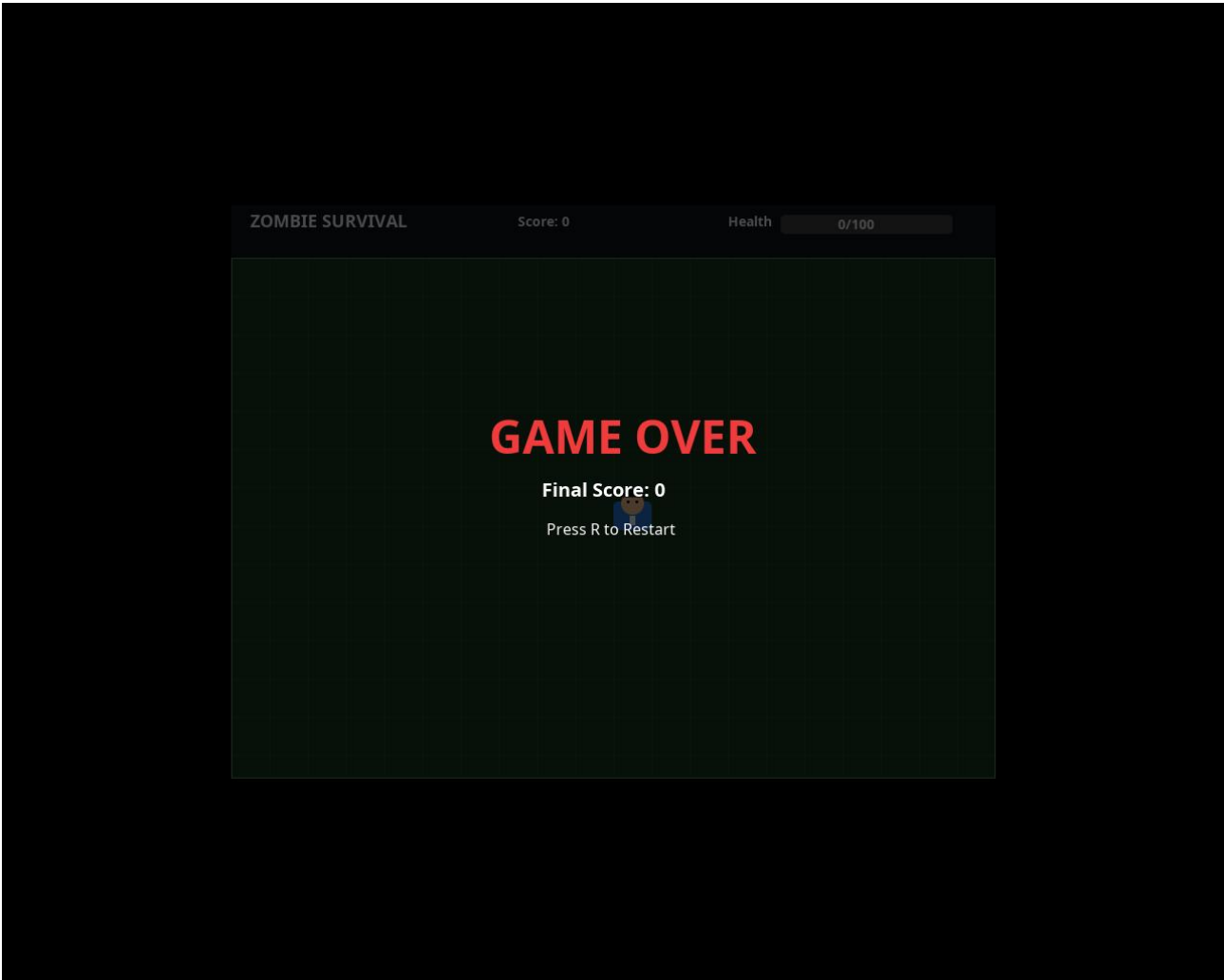


Figure 2: Game Over state with final score and restart instruction.

8. Final Code Structure

Class	Responsibility	Final Implementation
Main.java	Starts the application and creates the JFrame.	Launches the final game window.
GamePanel.java	Controls the game loop, rendering, input, spawning, collisions, score, health, Game Over and restart.	Main gameplay controller.
Player.java	Stores player state and handles movement, drawing and collision bounds.	Complete player implementation.
Zombie.java	Stores zombie state and handles movement, drawing and collision bounds.	Complete zombie implementation.
Bullet.java	Stores bullet state and handles movement, drawing and collision bounds.	Complete shooting implementation.

9. Final Game Flow

1. Start the game.
2. Player health is initialized to 100 and score to 0.
3. Zombie spawning begins with a 2-second interval.
4. Player moves using W/A/S/D and shoots using SPACE.
5. Zombies move toward the player.
6. When a bullet hits a zombie, the zombie and bullet are removed and score increases by 1.
7. At every 10 points, the zombie spawn interval becomes shorter, down to a minimum of 0.5 seconds.
8. When a zombie touches the player, health decreases.
9. When health reaches 0, the Game Over screen is displayed.
10. Press R to reset the player, score, zombies, bullets and spawning system.

10. Final Outcome

The 10-week development process has resulted in a functional 2D Zombie Survival Game built with Java Swing. The final implementation contains the major gameplay systems planned for the project: player movement, zombie spawning and pursuit, shooting, collision detection, scoring, health management, dynamic difficulty, Game Over, restart functionality and a visual HUD. The project was compiled and the final source structure was verified. The completed game is ready for final project submission and presentation.